# Bioinformatics 2026 – Project 1

## Assignment 1 – ORFs and k-mer frequencies (5 pts)

Write two Python scripts that take a FASTA file as input. Parse the sequence with Biopython (`Bio.SeqIO` / `Bio.Seq`). For testing, use the bacteriophage lambda genome (NC_001416.1) or any bacterial genome downloaded from NCBI.

### a) ORF finder – `orf_finder.py` (3 pts)

Write your own function that finds open reading frames in all six reading frames (three on the forward strand and three on the reverse complement). An ORF starts with ATG and ends at the first in-frame stop codon (TAA, TAG or TGA).

- For each ORF, report the frame, start and end position, length and the translated protein sequence. Give positions as 1-based coordinates of the original (forward) sequence, also for ORFs on the reverse strand.
- Add a minimum-length parameter.
- Biopython may be used for reading the file, reverse complement and translation, but not for finding the ORFs themselves.

Example call: `python orf_finder.py genome.fasta --min-len 100 --out orfs.csv`

### b) k-mer frequencies – `kmer_freq.py` (2 pts)

For a given k (passed as a command-line argument), count all k-mers in the sequence and report their counts and relative frequencies, sorted from the most frequent.

Example call: `python kmer_freq.py genome.fasta -k 3 --out kmers_k3.csv`

**Deliverables:** both scripts, the output tables for the chosen genome (`orfs.csv`, `kmers_k3.csv`) .

## Assignment 2 – an automated gene report from databases via API (5 pts)

Write a script `genereport.py` that, for a given gene symbol and organism, retrieves information from NCBI, UniProt and Ensembl through their public REST APIs and generates a report. It automates exactly what we did manually in the browser in Lab 2. Only `requests` is allowed (no Biopython and no ready-made wrappers such as `mygene`).

Example call: `python genereport.py APOE --organism "Homo sapiens" --out reports/`.

### Data sources and endpoints

| Database | API | What to retrieve |
|---|---|---|
| NCBI Gene | E-utilities: `esearch.fcgi` (db=gene), `esummary.fcgi` | Gene ID, full name, chromosomal location, aliases, summary, MIM cross-references |
| NCBI Nucleotide | E-utilities: `elink.fcgi` (gene→nuccore), `efetch.fcgi` (rettype=gb / fasta) | RefSeq mRNA accession and version, length, last modification date, CDS coordinates, `protein_id`, `transl_table` |
| UniProtKB | `rest.uniprot.org/uniprotkb/search` (format=json, `fields` parameter) | Accession, status (Reviewed/Unreviewed), protein name, function, length, ligands/cofactors, diseases, PDB structures, keywords |
| Ensembl | `rest.ensembl.org/lookup/symbol/{species}/{symbol}?expand=1` | Ensembl Gene ID, coordinates, number of transcripts, canonical transcript, biotype |

Documentation: NCBI E-utilities, UniProt REST API, Ensembl REST.

## Assignment 3 – RNA-seq analysis of lung adenocarcinoma (10 pts, 2 pts per task)

Dataset: GEO GSE40419 (Seo et al., 2012) – RNA-seq of 87 lung adenocarcinomas and 77 matched normal lung tissues (164 samples), normalised RPKM matrix in the series supplementary file.

1. Load the RPKM matrix and sample metadata (tissue, patient, sex, smoking, driver mutation), apply log2(RPKM + 1), filter out lowly expressed genes and check sample quality with a sample–sample correlation heatmap.
2. Visualise the samples with PCA, t-SNE and UMAP coloured by tissue and the clinical variables, and compare the three methods using the silhouette score.
3. Cluster the samples with k-means and hierarchical clustering and measure agreement with the tissue labels (ARI). Repeat for tumours only and test whether the clusters are associated with smoking or EGFR/KRAS status.
4. Find genes differentially expressed between tumour and normal tissue using a paired test with FDR correction and show them on a volcano plot. Compare the results with GSE10072 from Lab 3.
5. Train a tumour-vs-normal classifier with patient-grouped cross-validation (no data leakage) and report the ROC AUC and the most important genes. Test the model on GSE10072.